School of Future Tech

Case Study Report

on

MediTravel: Medical Tourism Portal (Healora)

by

Aman Ashok Koli

150096725119

1. Introduction to the Case Study

Medical tourism is a rapidly growing global industry where patients travel abroad for affordable, high-quality medical care. However, patients often struggle to find transparent pricing, reliable doctors, and accurate recovery information in a single, unified platform. This case study focuses on designing and implementing a fully responsive multi-page website called Healora, a medical tourism portal. The case study demonstrates how real-world web data can be structured using semantic HTML, styled responsively with CSS, and made interactive using JavaScript. It highlights the importance of user experience, structured data presentation, and interactive filtering in building a practical and reliable system for international patients.

2. Problem Statement / Case Background (Abstract)

Background

International patients face significant challenges when exploring treatment packages, comparing healthcare costs across borders, and connecting with verified hospitals. Traditional static websites are not sufficient to handle dynamic cost comparisons or filter treatments based on user preferences.

Abstract

This case study presents the design and implementation of the MediTravel platform (Healora) using front-end web technologies. A fully responsive, 5-page website was developed comprising Home, Treatments, Compare, Doctors, and Contact pages. The portal features structured cards with proper label-input associations, dropdown menus for country selection, and interactive doctor profiles. The system is evaluated based on its UI enhancements (hover transitions, keyframe animations) and functional JavaScript requirements (dynamic cost filters, form validation), showing that modern web tools can effectively streamline global healthcare decisions.

3. Case Study Design

The Healora portal is designed as a modular web architecture with the following stages:

- **Website Structure:** A 5-page interconnected layout. index.html serves as the landing page with hero sections and sliders. treatments.html holds package

cards with price and recovery data. compare.html houses the cost comparison tool. doctors.html displays specialist profiles, and contact.html contains the consultation form.

- **UI & CSS Design:** A consistent healthcare-themed palette (blues and greens like #0F4C75 and #00B4D8) is used. Layouts are strictly managed using CSS Flexbox for navigation and CSS Grid for treatment catalogs.

- **Interactive JavaScript Features:** Dedicated scripts (compare.js, contact.js, treatment.js, script.js) manage form validation, filter treatment visibility based on user dropdown selections, and generate side-by-side cost comparison tables dynamically.

4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study

Key methods and algorithms applied include:

- **Responsive Web Design Methods:**

    - CSS Flexbox for one-dimensional alignment (navbars, card internals).

    - CSS Grid for multi-column layouts, adapting from 4 columns to 1 column on smaller screens. * Media Queries (max-width: 1024px, 768px, 480px) used to stack layouts vertically and make buttons full-width in mobile view.

- **JavaScript Functional Algorithms:**

    - **Form Validation:** Using addEventListener("submit") and e.preventDefault() to check required fields, enforce email formatting (email.includes("@")), and dynamically output success/error messages.

    - **Dynamic Cost Comparison:** Extracting data from a nested JavaScript object (treatmentData) based on selected dropdown values and injecting it into the DOM via template literals (innerHTML).

    - **Real-time Filtering:** Iterating over node lists with querySelectorAll and matching dataset.country / dataset.specialty attributes to toggle the CSS display property.

Technology Stack Used for the Case

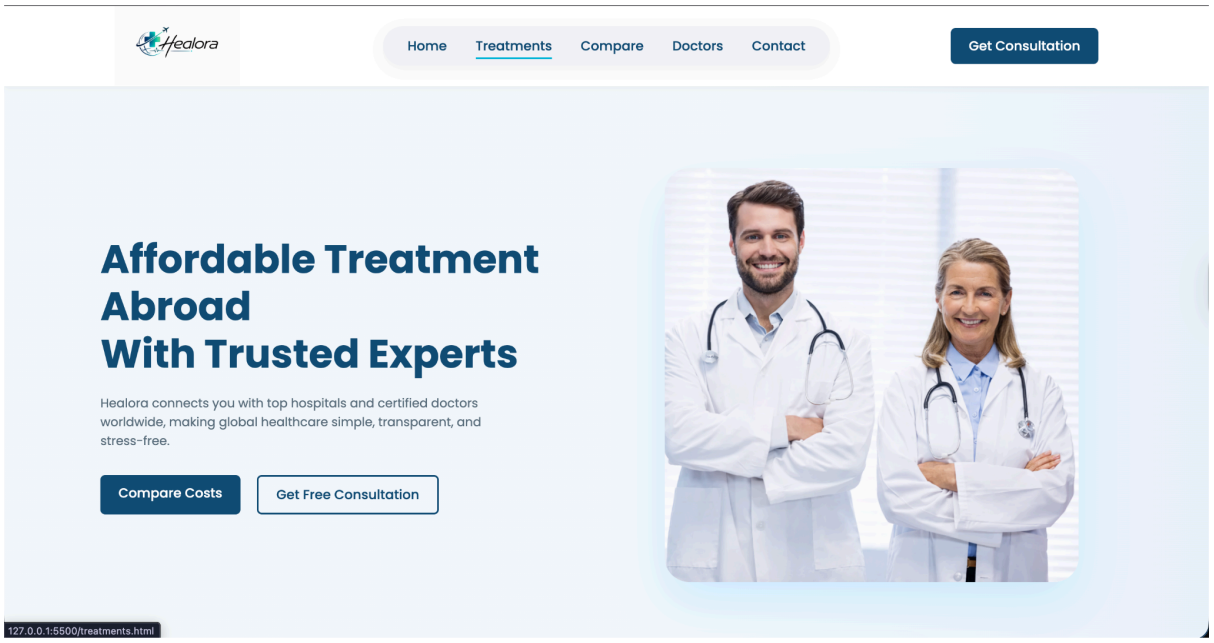
- Programming languages: HTML5, CSS3, JavaScript (ES6+).
- Environment: Visual Studio Code, Live Server, Git.

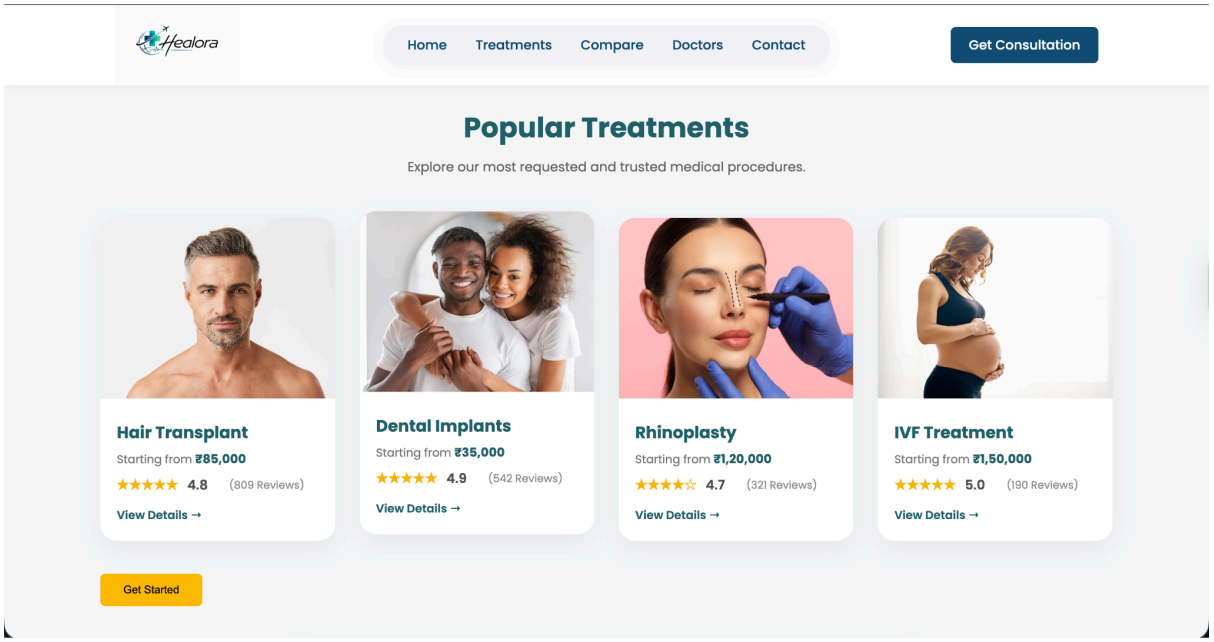
5. Problem Statement / Case Study Implementation Details and Snapshots.

The implementation is organized into specific script files for modularity:

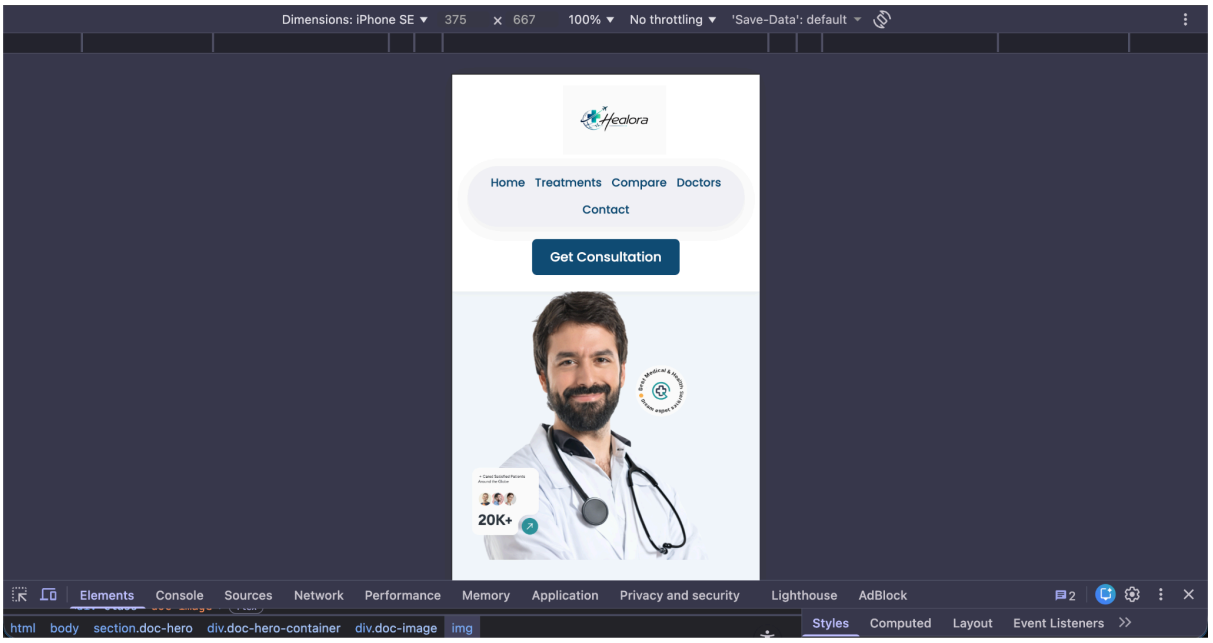
- compare.js: Contains the multi-dimensional treatmentData object and the compareTreatment() function to render HTML tables dynamically.
- contact.js: Handles the logic for the #consultForm, validating fullname, phone, and treatment radio buttons.
- treatment.js: Executes the filterCards() function to update the visibility of .treat-card elements based on #countryFilter and #specialtyFilter.
- style.css: Implements global styling, including @keyframes fadeUp animations, card transform: translateY(-8px) effects, and input focus highlights.

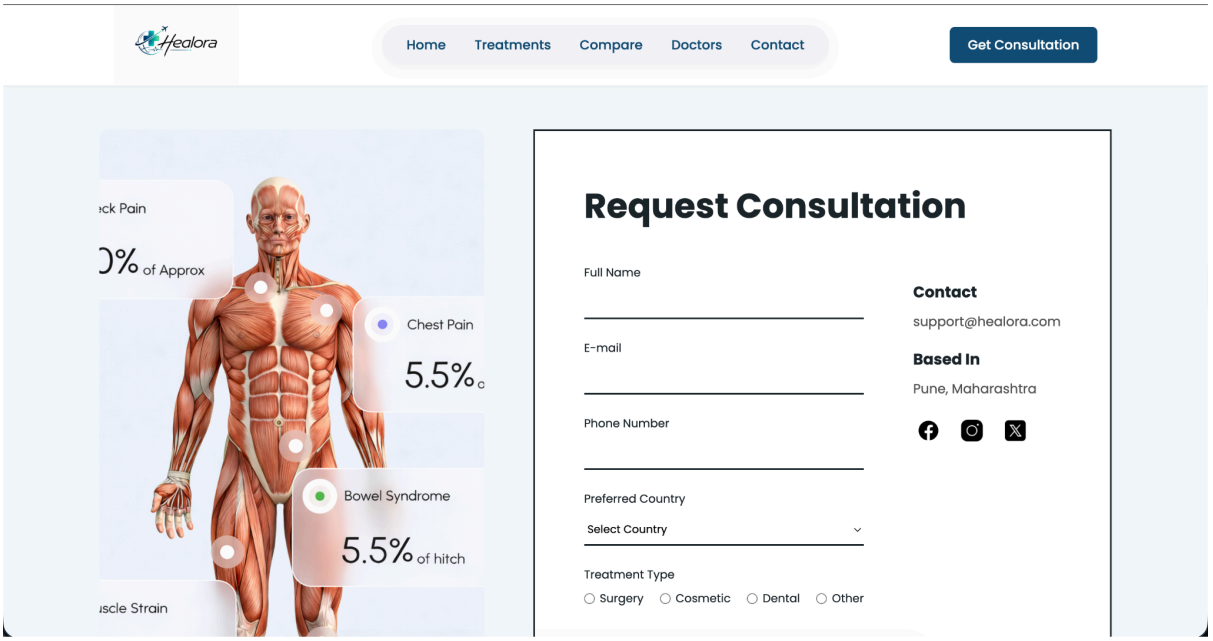
Snapshots you can include:





Mobile responsive view of the Doctors page





6. Problem Statement / Case Study Results and Conclusion.

Findings

- Proper application of CSS Grid and Flexbox significantly improves structural adaptability across desktop, tablet, and mobile interfaces.

- JavaScript-driven dynamic filtering provides a seamless user experience by updating data without requiring full page reloads.

- Input validation enhances data integrity for consultation requests before backend submission.

Conclusion

This case study demonstrates a complete front-end pipeline for building a Medical Tourism Portal. It successfully integrates structured semantic HTML, responsive CSS layouts, and functional JavaScript logic into a coherent system, fully satisfying the MediTravel problem statement and providing a strong foundation for future backend integration.

7. References

ITM Skills University Problem Statement: 85. MediTravel.

MDN Web Docs for HTML5 Semantic Tags, CSS Flexbox, and Grid documentation.
JavaScript ES6+ documentation for DOM manipulation methods (querySelector, getElementById, addEventListener).